Criterion C: Product Development

List of Techniques

- User-defined methods
- Evaluating User Input
- Error handling
- If-else Loops
- Nested Loops
- Data Processing
- File I/O (Input/Output)
- 2D Arrays
- Manipulation of Data
- Sorting Algorithm
- GUI/User Interface
- Lists
- Queue
- Hashmap

Program Overview

The overall structure of the program is a Java-based program that allows for users to create, view, and generate their own playlists. This is done through a music library with a hierarchical system that separates it from genres, artists, albums, and songs, which users are able to navigate through and select in order to build their playlists.

Algorithms

Class	Methods	Function
Main	main(String[] args}	Base of program, the starting point that will have a structure to reroute to methods depending on user input.
Generate	generatePlaylist()	Will generate a random playlist depending on user-defined length.
	getAllAvaliableSongs()	Will read through the available songs in the music library and turn them into a list.
	savePlayliststToFile(List)	Save the playlist to a list.
Create	createPlaylist()	Will direct the user to create their own playlist depending on the user-input.
	clearPlaylist()	Will clear the previously created playlist.
	removeSongFromPlaylist()	Removes added song from playlist.
	queueSong()	Add songs to a queue for the user to view.

	showQueue()	Queue will be outputted for the user to view.
View	loadPlaylists()	Playlists will load from a text file and then store them in a 2D array.
	displayPlaylists()	Displays the previously saved playlists in a user-friendly form

Playlist Creating and Song Selecting

```
public void createPlaylist() {
        System.out.print("Enter the name of your playlist: ");
        String playlistName = scanner.nextLine();

        while (true) {
            List<Genre> genres = new ArrayList<>(musicLibrary.values());
            System.out.println("\nAvailable Genres:");
            for (int i = 0; i < genres.size(); i++) {
                System.out.println((i + 1) + ". " + genres.get(i).getName());
            }
            System.out.println("0. Done");
            System.out.print("Choose a genre (number): ");
            int genreIndex = getValidInput(0, genres.size());
            if (genreIndex == 0) break;
            Genre selectedGenre = genres.get(genreIndex - 1);
            List<Artist> artists = new ArrayList<>(selectedGenre.getArtists().values());
            while (true) {
                System.out.println("\nArtists in " + selectedGenre.getName() + ":");
                for (int i = 0; i < artists.size(); i++) {
                    System.out.println((i + 1) + ". " + artists.get(i).getName());
                }
                System.out.println("0. Back");
                System.out.print("Choose an artist (number): ");
                int artistIndex = getValidInput(0, artists.size());
                if (artistIndex == 0) break;
                Artist selectedArtist = artists.get(artistIndex - 1);
                List<Album> albums = new ArrayList<>(selectedArtist.getAlbums().values());
                while (true) {
                    System.out.println("\nAlbums by " + selectedArtist.getName() + ":");
                    for (int i = 0; i < albums.size(); i++) {
                        System.out.println((i + 1) + ". " + albums.get(i).getName());
                    }
                    System.out.println("0. Back");
                    System.out.print("Choose an album (number): ");
                    int albumIndex = getValidInput(0, albums.size());
                    if (albumIndex == 0) break;
                    Album selectedAlbum = albums.get(albumIndex - 1);
                    List<Song> songs = selectedAlbum.getSongs();
                    while (true) {
                        System.out.println("\nSongs in " + selectedAlbum.getName() + ":");
                        for (int i = 0; i < songs.size(); i++) {
                            System.out.println((i + 1) + ". " + songs.get(i).getTitle());
                        }
                        System.out.println("0. Back");
                        System.out.print("Choose a song to add (number): ");
                        int songIndex = getValidInput(0, songs.size());
                        if (songIndex == 0) break;
```

```
                    Song selectedSong = songs.get(songIndex - 1);
                    playlist.add(selectedSong.getTitle());
                    System.out.println("'" + selectedSong.getTitle() + "' added to
your playlist!");
                }
            }
        }
    }
```

The goal of the createPlaylist() method is to allow the user to create custom playlists using a hierarchy of data, genres to artists to albums to songs. This ensures that the song selection process is efficient and user friendly which will allow for a good user experience.

The user-defined methods in the code, such as the getValidInput(), allows for the input to be confirmed as correct which improves the modularity of the program, allowing for flexibility and improved efficiency. Through evaluating the user input, it prevents the system from crashing or losing data from invalid inputs.

getValidInput() samples

```
int songIndex = getValidInput(0, songs.size());
                    if (songIndex == 0) break;
```

```
private int getValidInput(int min, int max) {
        int choice;
        while (true) {
            try {
                choice = Integer.parseInt(scanner.nextLine());
                if (choice >= min && choice <= max) {
                    return choice;
                } else {
                    System.out.print("Invalid choice, try again:
");
                }
            } catch (NumberFormatException e) {
                System.out.print("Invalid input, enter a number:
");
            }
        }
    }
```

Hierarchical Structure

The If-Else loops used in the program control the user navigation through a hierarchical selection which allows for efficiency and prevents user error when navigating the program. Even so, the use of nested loops allows for the implementation of the hierarchical structure, making it necessary to use for this program. The use of the nested loops within this structure is easily guidable for the user to select their choices. This structure also allows the user to return to their previous action which will improve user experience.

If-Else loop sample

```
while (true) {
            System.out.println("\n1. View a playlist\n2. Generate
a playlist\n3. Create a playlist\n4. Clear current playlist\n5.
Remove song from playlist\n6.Queue a song\n7. Show
Queue\n8.Exit");
            String decision = scanner.nextLine();
            if (decision.equals("1")) { // View existing
playlists
                View.displayPlaylists();
            }
            else if (decision.equals("2")) { // Generate a
playlist
                System.out.print("How many songs would you like
in your generated playlist? ");
                int playlistLength =
Integer.parseInt(scanner.nextLine());
                Generate objB = new Generate(objC); // Pass the
Create instance to Generate
                objB.generatePlaylist(playlistLength); //
Generate the playlist
            }
            else if (decision.equals("3")) { // Create a new
playlist
                objC.createPlaylist(); // Use the existing
instance of Create
            }
          }
        scanner.close();
    }
}
```

Array List

The class also requires the use of lists, Array List, in order to store the user section dynamically. This means that it would allow for the user to add or remove songs from their playlist in order to update it to their liking. This is necessary because the client requires a dynamic system because their taste of music is always changing.

```
public void removeSongFromPlaylist() {
       if (playlist.isEmpty()) {
           System.out.println("Your playlist is empty.");
           return;
       }
       System.out.println("Current Playlist:");
       for (int i = 0; i < playlist.size(); i++) {
           System.out.println((i + 1) + ". " + playlist.get(i));
       }
       System.out.print("Enter the number of the song to remove: ");
       int songIndex = getValidInput(1, playlist.size()) - 1; // Adjust for zero-based
index
       String removedSong = playlist.remove(songIndex);
       System.out.println("'" + removedSong + "' has been removed from your playlist.");
   }
```

Sorting

In order to improve the user experience, organizing the artists in alphabetical order when selecting songs.

Sorting Sample	`public List<Artist> getSortedArtists() {`
	`  List<Artist> artistList = new ArrayList<>(artists.values());`
	`  Collections.sort(artistList, (a1, a2) -> a1.getName().compareToIgnoreCase(a2.getName()));`
	`  return artistList;`
	`  }`
	`}`

The primary technique used is the Collections.sort() method which organizes the artists by name efficiently using the O(b log n) method due to Collections.Sort(). This is necessary for the program as it accounts for scalability and dynamic. It is important to sort with a large dataset because without it, it will cause confusion and limit the user experience.

Data processing is also crucial in the method as it converts the HashMap values into a list that allows for the program to process the data. This is done through List<Artist>.

Hashmap Sample	`public List<Artist> getSortedArtists() {`
	`  List<Artist> artistList = new ArrayList<>(artists.values());`
	`  Collections.sort(artistList, (a1, a2) -> a1.getName().compareToIgnoreCase(a2.getName()));`
	`  return artistList;`
	`  }`
	`}`

File I/O

By using the file i.o, it allows for the user-created playlists to be saved into a file after the user created the playlist. This can be done through the BufferWriter/Filewriter. Buffer Writer has the advantages of reducing the disk write operations which makes it faster and can handle large playlist files easily and efficiently. Because of this, I found it was a good fit for my program.

File I/O sample

```
private void savePlaylistToFile(String playlistName) {
       try (BufferedWriter writer = new BufferedWriter(new
FileWriter(FILE_NAME, true))) {
           writer.write("Playlist: " + playlistName);
           writer.newLine();
           writer.write("==========================");
           writer.newLine();
           for (String song : playlist) {
               writer.write(song);
               writer.newLine();
           }
           writer.write("--------------------------");
           writer.newLine();
           // Ensure everything is written
           writer.flush();
           System.out.println("Playlist saved to: " +
FILE_NAME);
       } catch (IOException e) {
           System.out.println("An error occurred while saving
the playlist.");
           e.printStackTrace();
       }
   }
```

The try-catch method prevents the program from crashing if playlist is not saved properly.

T-Catch method

```
 private void savePlaylistToFile(List<String> playlist) {
       try {
           Files.write(Paths.get("playlists.txt"), playlist); //
Write the playlist to playlists.txt
           System.out.println("Playlist saved to
playlists.txt");
       } catch (IOException e) {
           System.err.println("Error saving playlist: " +
e.getMessage());
       }
   }
}
```

This technique is also used in the loadPlaylists() and displayPlaylists() method which loads the saved playlist from a saved file to display them to the user. This allowed for the user to view their previous playlists, contributing to the client's request.

Playlist Generation

The Generate class is unique as it uses all available songs from the music library and uses the user-inputs specified number of songs, to create a playlist.

This class uses data extraction, as it collects the songs from the user's library and stores them into a list as well as confirms that each song is unique, ensuring no repeats. This is required in order to process the data into a usable and dynamic format which generates playlists.

The class also required the method savePlaylistToFile(generatedPlaylist); which allowed for data storage which allows the user to save their generated playlist.

2D Arrays

The loadPlaylist() method in the View class stores playlists using a collection of songs using a 2D array. The 2D arrays are able to separate the results into columns and arrays which makes them easier to display, which is why they were necessary for this program. This represents a playlist using rows and each column represents the songs within the playlist. This is a necessary technique as it allows for an efficient data structure, contributing to a better user experience.

2D Array Samples	

	`public static String[][] loadPlaylists() {`
	`  File file = new File(FILE_NAME);`
	
	`  if (!file.exists() || file.length() == 0) {`
	`  System.out.println("No playlists found.");`
	`  return new String[0][0]; // Return an empty 2D array`
	`  }`
	`  List<String[]> playlists = new ArrayList<>();`

	`// Convert List<String[]> to String[][]`
	`  String[][] result = new String[playlists.size()][];`
	`  for (int i = 0; i < playlists.size(); i++) {`
	`  result[i] = playlists.get(i);`
	`  }`
	`  return result;`
	`  }`

	`// Method to display the playlists`
	`  public static void displayPlaylists() {`
	`  String[][] playlists = loadPlaylists();`
	`  if (playlists.length == 0) {`
	`  System.out.println("No playlists found.");`
	`  return;`
	`  }`
	`  System.out.println("\n=== Your Saved Playlists ===\n");`
	`  for (int i = 0; i < playlists.length; i++) {`
	`  System.out.println("Playlist " + (i + 1) + ":");`
	`  for (String song : playlists[i]) {`
	`  if (song != null) { // Check for null to avoid printing empty slots`

```
                    System.out.println(" - " + song);
                }
            }
            System.out.println("--------------------------");
        }
        System.out.println("\n==========================");
    }
}
```

Queue

The methods queueSong(), showQueue in the create class allow the user to create a queue of songs to test the playlist or song choice. This is necessary for the request of the client, who requested this functionality. The FIFO order mimics the functionality of common streaming services. This method was necessary in order to implement the queue function.

Queue Sample

```
public void queueSong() {
        System.out.println("Select a song to queue:");
        for (int i = 0; i < playlist.size(); i++) {
            System.out.println((i + 1) + ". " + playlist.get(i));
        }
        System.out.print("Enter the number of the song to queue:
");
        int songIndex = getValidInput(1, playlist.size()) - 1; //
Adjust for zero-based index
        String songToQueue = playlist.get(songIndex);
        songQueue.enqueue(songToQueue);
        System.out.println("'" + songToQueue + "' has been added
to the queue.");
    }
    public void playNextSong() {
        if (songQueue.isEmpty()) {
            System.out.println("No songs in the queue to play.");
            return;
        }
        String songToPlay = songQueue.dequeue();
        System.out.println("Now playing: " + songToPlay);
    }
    public void showQueue() {
        songQueue.display();
    }
```

Words: 732